

Manual de Despliegue — Frontend y Backend — Proyecto HeartCoin

Introducción

Este documento constituye el manual de despliegue del proyecto HeartCoin, cubriendo tanto la aplicación cliente (Flutter) como el backend (Supabase: base de datos, secrets y Edge Functions). Describe los comandos, requisitos y el orden de operaciones necesarios para desplegar cada capa, basado en el procedimiento efectivamente utilizado durante el desarrollo del proyecto, no en un procedimiento teórico no verificado.

Objetivo

Proporcionar una guía operativa completa y verificada para desplegar HeartCoin, tanto el backend sobre Supabase como el cliente Flutter, incluyendo la gestión de credenciales y secretos, y dejar constancia de qué partes de este proceso están actualmente automatizadas y cuáles requieren ejecución manual.

Alcance

Este documento cubre el despliegue del backend (migraciones de base de datos, configuración de secrets, publicación de Edge Functions) y la compilación del frontend para al menos una plataforma de escritorio, verificada de forma real durante este proyecto. No cubre el proceso de publicación en tiendas de aplicaciones (Google Play / App Store), dado que no fue ejecutado ni verificado en el periodo cubierto por este trabajo, ni la configuración de infraestructura de Supabase en sí (creación de una nueva organización o proyecto desde cero).

Requisitos Previos

Requisito	Uso
Flutter SDK (canal estable)	Compilación y ejecución del frontend
Un dispositivo o emulador Android/iOS, o Windows como plataforma de escritorio	Ejecución/compilación del cliente
Supabase CLI	Migraciones, gestión de secrets, despliegue de Edge Functions

Cuenta de Supabase con acceso al proyecto (supabase login)	Autenticación del CLI
Python 3 (opcional)	Únicamente si se requiere generar documentación en PDF a partir de Markdown, no forma parte del despliegue de la aplicación en sí

1. Manual de Despliegue del Backend (Supabase)

1.1 Vinculación del proyecto

El proyecto de Supabase ya está enlazado localmente. Para vincularlo en una máquina nueva:

```
supabase login supabase link --project-ref bamrvwzpcqwouwbvomw
```

1.2 Migraciones de Base de Datos

Estado actual del proceso: las migraciones de este proyecto se aplicaron mediante archivos `.sql` individuales ejecutados directamente contra el proyecto vinculado, **no mediante el sistema de migraciones versionadas de Supabase** (`supabase/migrations/`). Cada cambio de esquema se aplicó con:

```
supabase db query --linked -f ruta/al/archivo.sql
```

Este método fue efectivo para el desarrollo iterativo del proyecto, pero **no deja un historial de migraciones versionado y reproducible automáticamente** en un entorno nuevo — se documenta como área de mejora en la sección correspondiente.

1.3 Gestión de Secrets

Cada integración externa requiere sus propios secrets, configurados una sola vez por ambiente:

```
# MediaAAS (almacenamiento de archivos) supabase secrets set MEDIAAS_API_HOST=...
MEDIAAS_API_KEY=... MEDIAAS_SECRET_KEY=... # Original Auth (verificación de correo) –
dos ambientes distintos supabase secrets set
ORIGINAL_AUTH_BASE_URL=https://api.originalauth.com
ORIGINAL_AUTH_API_KEY=oa_live_<key_produccion> # o, para ambiente de prueba: supabase
secrets set ORIGINAL_AUTH_BASE_URL=https://api.test.originalauth.com
ORIGINAL_AUTH_API_KEY=oa_live_<key_test> # Original Pay (pagos con HC) supabase secrets
set ORIGINALPAY_API_KEY=heartcoin_live_<key_generada_por_heartcoin> supabase secrets set
ORIGINALPAY_WEBHOOK_TOKEN_APP=app... ORIGINALPAY_WEBHOOK_SECRET_KEY=sk...
```

`SUPABASE_SERVICE_ROLE_KEY` **no requiere configuración manual:** Supabase la inyecta automáticamente en el entorno de ejecución de toda Edge Function del proyecto.

1.4 Despliegue de Edge Functions

Cada función se despliega de forma individual:

```
supabase functions deploy upload-media supabase functions deploy delete-account supabase
functions deploy originalauth-register supabase functions deploy
originalauth-verify-email supabase functions deploy originalauth-resend-verification
supabase functions deploy originalpay-balance --no-verify-jwt supabase functions deploy
originalpay-redeem --no-verify-jwt supabase functions deploy originalpay-webhook
--no-verify-jwt
```

Nota crítica de despliegue: las tres funciones de Original Pay requieren el flag `--no-verify-jwt`, dado que son invocadas por un tercero externo sin sesión de Supabase. Omitir este flag provoca que la plataforma rechace toda petición con `401 UNAUTHORIZED_NO_AUTH_HEADER` antes de que el código de la función siquiera se ejecute — un error de despliegue real detectado y corregido durante este proyecto.

Cada función que depende de un módulo compartido (`_shared/originalauth.ts`, `_shared/originalpay_auth.ts`) lo incluye automáticamente en el paquete desplegado; no requiere un paso de "build" o empaquetado separado.

1.5 Verificación Post-Despliegue

Se recomienda, después de cada despliegue, ejecutar al menos una petición de verificación real contra la función desplegada (no únicamente confirmar que el comando de despliegue terminó sin error), dado que un despliegue exitoso a nivel de plataforma no garantiza que la lógica interna de la función esté correctamente configurada (por ejemplo, un secret faltante o un ambiente incorrecto no provoca un error de despliegue, pero sí una falla en tiempo de ejecución).

2. Manual de Despliegue del Frontend (Flutter)

2.1 Configuración de Credenciales

Las credenciales de conexión a Supabase (URL del proyecto y `anon key`) están definidas directamente en `lib/main.dart`. Al preparar un nuevo entorno de despliegue (por ejemplo, un ambiente de staging separado), este archivo debe actualizarse manualmente con la URL y clave correspondientes a ese proyecto de Supabase.

2.2 Instalación de Dependencias

```
flutter pub get
```

2.3 Ejecución en Desarrollo

```
flutter run
```

2.4 Compilación para Distribución

Verificado de forma real durante este proyecto:

```
flutter build windows --debug
```

Este comando fue ejecutado y confirmado exitoso, generando un ejecutable funcional en `build\windows\x64\runner\Debug\heartcoin.exe`.

No verificado en este trabajo (comandos estándar de Flutter, no ejecutados ni confirmados durante este proyecto):

```
flutter build apk --release # Android flutter build appbundle --release # Android  
(Google Play) flutter build ios --release # iOS (requiere macOS + Xcode)
```

2.5 Íconos de la Aplicación

El proyecto utiliza el paquete `flutter_launcher_icons`, configurado en `pubspec.yaml`, para generar los íconos de la aplicación en todas las resoluciones necesarias a partir de una única imagen fuente (`assets/logo_heartcoin_vista.png`). Se ejecuta como un paso independiente antes de la compilación final:

```
flutter pub run flutter_launcher_icons
```

2.6 Publicación en Tiendas de Aplicaciones

No cubierto por este manual. El proceso de firma de la aplicación (keystore de Android, certificados de distribución de iOS) y de publicación en Google Play Store / Apple App Store no fue ejecutado durante este proyecto y queda fuera del alcance verificado de este documento.

3. Orden de Despliegue y Dependencias entre Capas

El backend debe desplegarse **antes** que cualquier versión del frontend que dependa de sus cambios, dado que el frontend consume la API generada por el esquema y las Edge Functions del backend. En la práctica de este proyecto, el orden seguido fue:

1. Cambios de esquema (migraciones .sql vía supabase db query)
2. Secrets nuevos o actualizados (supabase secrets set)
3. Despliegue de Edge Functions afectadas (supabase functions deploy)
4. Verificación real del endpoint desplegado
5. Cambios correspondientes en el cliente Flutter (si aplica)
6. flutter analyze + flutter build (verificación del cliente)

Dado que Supabase no versiona el esquema mediante el flujo de migraciones estándar en este proyecto (ver 1.2), **no existe garantía automática de que un ambiente nuevo pueda reconstruirse ejecutando un solo comando** — el orden anterior debe reproducirse manualmente a partir del historial de archivos `.sql` utilizados.

Problemas Detectados

1. **Ausencia de un pipeline de CI/CD:** la totalidad de los despliegues de este proyecto se ejecutaron mediante comandos manuales del CLI de Supabase y de Flutter, sin automatización que garantice consistencia entre ejecuciones o entornos.
2. **Migraciones de base de datos no versionadas mediante el mecanismo estándar de Supabase:** el historial de cambios de esquema vive en archivos `.sql` dispersos, no en el directorio `supabase/migrations/` que permitiría reconstrucción automática de un ambiente nuevo con `supabase db push`.
3. **Credenciales de Supabase hardcodeadas en `lib/main.dart`:** no existe un mecanismo de variables de entorno o configuración por ambiente (desarrollo/staging/producción) para el cliente Flutter, requiriendo edición manual del código fuente para apuntar a un proyecto distinto.
4. **El proceso de compilación para plataformas móviles (Android/iOS) y su publicación en tiendas no fue verificado** durante este trabajo, únicamente la compilación de escritorio (Windows).

Riesgos e Impactos

- El riesgo del punto 1 es de **impacto alto en consistencia y trazabilidad:** sin CI/CD, la calidad de un despliegue depende de que la persona que lo ejecuta manualmente siga correctamente cada paso documentado en este manual, sin garantía automatizada.
- El riesgo del punto 2 es de **impacto alto ante la necesidad de reconstruir el proyecto en un ambiente nuevo** (por ejemplo, un ambiente de staging separado del de producción): reconstruir el esquema completo requeriría localizar y reejecutar manualmente cada archivo `.sql` histórico, en el orden correcto.
- El riesgo del punto 3 es de **impacto medio en seguridad y en agilidad operativa:** aunque la `anon key` es pública por diseño y no representa una fuga de credenciales sensibles, la ausencia de configuración por ambiente incrementa el riesgo de desplegar accidentalmente una compilación apuntando al proyecto incorrecto.
- El riesgo del punto 4 es de **impacto en confianza sobre la preparación real para producción móvil:** no puede garantizarse, a partir de este trabajo, que el proceso de compilación y publicación para Android/iOS esté libre de fricciones no anticipadas.

Fortalezas Identificadas

- **Procedimiento de despliegue de Edge Functions simple y consistente**, con un patrón uniforme (`supabase functions deploy <nombre>`) aplicado a las ocho funciones del proyecto.
- **Verificación real post-despliegue practicada de forma disciplinada** durante este proyecto, no solo confirmación de que el comando terminó sin error — práctica que permitió detectar el defecto real del flag `--no-verify-jwt` antes de que afectara a un usuario.
- **Gestión de secretos centralizada** mediante el mecanismo nativo de Supabase, sin credenciales sensibles almacenadas en el repositorio de código.
- **Compilación de escritorio verificada de forma real** como mecanismo de confirmación de que el código no solo analiza correctamente, sino que compila a un binario funcional.

Áreas de Mejora

- Migrar el historial de cambios de esquema al sistema de migraciones versionadas de Supabase (`supabase/migrations/`), permitiendo reconstrucción reproducible de un ambiente nuevo.
- Incorporar variables de entorno o un mecanismo de configuración por ambiente (`flutter run --dart-define`) para las credenciales de Supabase del cliente, eliminando la necesidad de editar código fuente para apuntar a un proyecto distinto.
- Establecer un pipeline de CI/CD, al menos para el despliegue de Edge Functions, dado que es el componente con el ciclo de actualización más frecuente observado durante este proyecto.
- Verificar y documentar el proceso completo de compilación y publicación para Android e iOS antes de cualquier lanzamiento real a tiendas de aplicaciones.

Recomendaciones

1. Priorizar la migración al sistema de migraciones versionadas de Supabase, dado que es el hallazgo con mayor impacto ante cualquier necesidad futura de reconstruir el ambiente desde cero.
2. Adoptar `--dart-define` o un archivo de configuración por ambiente para las credenciales del cliente Flutter, como mejora de bajo esfuerzo y alto valor operativo.
3. Mantener, como estándar obligatorio para toda nueva Edge Function, la práctica ya adoptada de verificación real post-despliegue antes de considerar el despliegue completo.

Conclusión

El proceso de despliegue de HeartCoin, aunque actualmente manual en su totalidad, está bien documentado y ha sido ejecutado de forma repetida y consistente a lo largo del proyecto, con una práctica disciplinada de verificación real post-despliegue que ha permitido detectar y corregir defectos de configuración antes de que llegaran a producción de forma no controlada. Las principales brechas identificadas —ausencia de CI/CD, migraciones no versionadas y configuración de credenciales

hardcodeada— son mejoras de madurez operativa esperables en un proyecto de este tamaño y etapa, no defectos que comprometan la funcionalidad actual del sistema.

Anexo: Checklist de Despliegue

Backend: - ☐ `supabase login / supabase link` (solo en máquina nueva) - ☐ Aplicar migraciones `.sql` pendientes vía `supabase db query --linked -f` - ☐ Configurar/actualizar secrets necesarios (`supabase secrets set`) - ☐ Desplegar Edge Functions afectadas (`supabase functions deploy`, con `--no-verify-jwt` si aplica) - ☐ Verificar cada función desplegada con una petición real, no solo confirmar el despliegue

Frontend: - ☐ Confirmar que `lib/main.dart` apunta al proyecto de Supabase correcto - ☐ `flutter pub get` - ☐ `flutter analyze` (debe estar limpio antes de compilar) - ☐ `flutter build <plataforma>` - ☐ (Pendiente de verificar) Firma y publicación en tienda correspondiente